

МФТИ

Алгоритмы и структуры данных II

Семинар 03

Динамическое программирование (3)

Весна 2026

Строгие подробные решения с доказательствами

Содержание

Обозначения и соглашения	2
1 Задача 1. Минимальный гамильтонов цикл	3
2 Задача 2. Гамильтоновы циклы и пути в неориентированном графе	4
3 Задача 3. Преобразования по подмаскам и надмаскам	6
4 Задача 4. Подмножество с суммой не больше единицы	7
5 Задача 5. Минимальная линейная укладка графа	8
6 Задача 6. Встречи и настроение	9
7 Задача 7. Замощение доски доминошками	10
8 Задача 8. Двухцветные раскраски без монохромных троек	12
9 Задача 9. Разрезание таблицы $2 \times n$ на тройки	13
10 Задача 10. Покупка рюкзаков	14
11 Задача 11. Хроматическое число	16
12 Задача 12. Минимальное число пакетов	18
13 Задача 13. Интересные тройки шестигранных кубиков	19

Обозначения и соглашения

Во всех оценках n обозначает размер входа, если в условии конкретной задачи не сказано иначе. Значение $+\infty$ используется для недостижимого состояния в задачах минимизации, а $-\infty$ — в задачах максимизации. Слово «путь» может означать маршрут с повторениями вершин только там, где это прямо следует из постановки; во всех остальных случаях путь простой.

1. Задача 1. Минимальный гамильтонов цикл

Решение. Зафиксируем стартовую вершину $s = 0$. Для множества $S \subseteq V$, содержащего s , и $v \in S$ определим

$$dp[S][v] = \min\{\text{вес пути из } s \text{ в } v, \text{ который проходит ровно по } S\}.$$

Каждая вершина в таком пути используется один раз. Начальное состояние $dp[\{s\}][s] = 0$, остальные значения равны $+\infty$. Переход:

$$dp[S][v] = \min_{u \in S \setminus \{v\}} (dp[S \setminus \{v\}][u] + w(u, v)). \quad (1)$$

Отсутствующему ребру приписываем вес $+\infty$. После заполнения таблицы ответ равен

$$\min_{v \neq s} (dp[V][v] + w(v, s)). \quad (2)$$

Для восстановления цикла в каждом конечном состоянии сохраняется минимизирующий предок u .

Доказательство корректности. Докажем смысл $dp[S][v]$ индукцией по $|S|$. Для $S = \{s\}$ существует единственный пустой путь веса 0. В любом пути, учитываемом состоянием (S, v) , предпоследняя вершина есть некоторый $u \in S \setminus \{v\}$. После удаления последнего ребра остаётся путь, проходящий ровно по $S \setminus \{v\}$, поэтому его вес не меньше соответствующего значения в (1). Обратно, к любому конечному кандидату (1) можно дописать ребро (u, v) ; вершина v до этого не использовалась. Таким образом, переход и не занижает, и достигает оптимум.

Удаление из гамильтонова цикла последнего ребра в s даёт один из путей в (2), а добавление ребра (v, s) к любому конечному кандидату (2) даёт гамильтонов цикл. Поэтому (2) в точности равно искомому минимуму. $\square$

Сложность. Состояний $\mathcal{O}(2^n n)$, переход перебирает до n предков: $\mathcal{O}(2^n n^2)$ времени и $\mathcal{O}(2^n n)$ памяти.

2. Задача 2. Гамильтоновы циклы и пути в неориентированном графе

Во всём решении битовое множество соседей вершины v обозначается N_v . Считаем, что операция над n -битовой маской занимает $O(1)$, как обычно в анализе динамики по маскам.

Пункт Существование гамильтонова цикла

Решение. Зафиксируем вершину r . Для каждого $S \ni r$ храним n -битовую маску $P[S]$: бит v установлен, если в $G[S]$ существует гамильтонов путь из r в v . База $P[\{r\}] = \{r\}$. Для $v \in S \setminus \{r\}$

$$v \in P[S] \iff (P[S \setminus \{v\}] \& N_v) \neq 0. \quad (3)$$

Гамильтонов цикл существует тогда и только тогда, когда

$$(P[V] \& N_r) \neq 0. \quad (4)$$

Доказательство корректности. В гамильтоновом пути из r в v предпоследняя вершина u смежна с v , а удаление v оставляет гамильтонов путь в $S \setminus \{v\}$. Это доказывает необходимость (3). Для достаточности берём любой установленный бит u в пересечении и дописываем ребро uv к соответствующему пути. Индукция по $|S|$ доказывает смысл всех масок.

Если конец гамильтонова пути из r смежен с r , замыкающее ребро даёт цикл. Если цикл существует, удаление одного из двух рёбер цикла, инцидентных r , даёт путь, удовлетворяющий (4). Следовательно, критерий точен. $\square$

Пункт Ответ для каждой пары концов

Решение. Те же маски $P[S]$ позволяют не запускать динамику отдельно из каждой вершины. Создадим для каждого u битовую маску $Ans[u]$ возможных вторых концов.

Переберём все $X \subseteq V$, содержащие r , и положим

$$Y = (V \setminus X) \cup \{r\}.$$

Для каждого установленного бита $u \in P[X]$ выполним

$$Ans[u] \leftarrow Ans[u] \mid P[Y]. \quad (5)$$

Симметрию можно добавить явно: если $v \in Ans[u]$, то установить $u \in Ans[v]$.

Доказательство корректности. Пусть $u \in P[X]$ и $v \in P[Y]$. Первый путь идёт из r в u и использует ровно X , второй — из r в v и использует ровно Y . Множества пересекаются только в r . Развернув первый путь и приклеив к нему второй, получаем простой путь из u в v , который проходит по всем вершинам ровно один раз. Поэтому алгоритм не создаёт ложных ответов.

Обратно, возьмём произвольный гамильтонов путь из u в v . Вершина r делит его на две части. Пусть X состоит из вершин участка от u до r , а Y — от r до v . Тогда $X \cap Y = \{r\}$,

$X \cup Y = V$, $u \in P[X]$ и $v \in P[Y]$. Соответствующая итерация (5) обязательно установит ответ для пары (u, v) . Случай, когда r является концом, покрывается множеством $X = \{r\}$ или $Y = \{r\}$. $\square$

Сложность. Для каждой из 2^{n-1} масок формула (3) проверяется не более чем для n концов: $\mathcal{O}(2^n n)$ времени. В (5) перебирается не более n битов для каждой маски, а объединение масок занимает $\mathcal{O}(1)$; это ещё $\mathcal{O}(2^n n)$. Память $\mathcal{O}(2^n + n)$ машинных слов.

3. Задача 3. Преобразования по подмаскам и надмаскам

Решение. Все четыре задачи решаются вариантом быстрого преобразования по подмножествам.

Пункт Максимум по надмаскам

Инициализируем $f[mask] = a[mask]$. Для каждого бита $b = 0, \dots, n - 1$ и каждой маски без этого бита выполняем

$$f[mask] \leftarrow \max(f[mask], f[mask \cup \{b\}]). \quad (6)$$

После всех битов $f[mask] = \max_{T \supseteq mask} a[T]$.

Пункт Максимум по подмаскам

Для маски, содержащей b , выполняем

$$f[mask] \leftarrow \max(f[mask], f[mask \setminus \{b\}]). \quad (7)$$

Пункт Сумма по надмаскам и подмаскам

В (6) и (7) заменяем $\max$ на сложение. Получаются соответственно

$$f[mask] = \sum_{T \supseteq mask} a[T], \quad f[mask] = \sum_{T \subseteq mask} a[T].$$

Доказательство корректности. После обработки первых q битов в варианте надмасок $f[mask]$ агрегирует значения тех надмасок, которые могут отличаться от $mask$ только среди этих q битов; остальные установленные сверх $mask$ биты ещё не разрешены. Переход разбивает варианты на два непересекающихся класса: бит b не добавлен и бит b добавлен. Для максимума берётся лучший класс, для суммы классы складываются. Индукция по q даёт требуемый результат после n битов. Для подмасок доказательство идентично с удалением бита. $\square$

Сложность. Каждая пара «бит — маска» обрабатывается один раз: $\mathcal{O}(n2^n)$ времени и $\mathcal{O}(2^n)$ памяти для каждого преобразования.

4. Задача 4. Подмножество с суммой не больше единицы

Решение. Разобьём индексы на две части L, R размеров не более $\lceil n/2 \rceil$. Перебором всех подмножеств построим списки сумм

$$X = \left\{ \sum_{i \in A} a_i : A \subseteq L \right\}, \quad Y = \left\{ \sum_{i \in B} a_i : B \subseteq R \right\}.$$

Вместе с суммой сохраним маску подмножества. Отсортируем Y . Для каждого $x \in X$ бинарным поиском найдём наибольшее $y \in Y$, удовлетворяющее $y \leq 1 - x$. Среди всех найденных пар выберем максимальное $x + y$, а ответное множество восстановим объединением масок. Если $x > 1$, его можно пропустить, поскольку все $a_i \geq 0$.

Доказательство корректности. Каждое подмножество $S \subseteq \{1, \dots, n\}$ единственным образом представляется как $A \cup B$, где $A \subseteq L$, $B \subseteq R$. При фиксированном A с суммой x допустимость требует $\sum_{i \in B} a_i \leq 1 - x$, а бинарный поиск выбирает наибольшую возможную сумму правой части. Значит, для каждой левой части рассматривается лучший совместимый вариант. В частности, рассматривается левая часть глобально оптимального множества и выбирается правая сумма не меньше его правой суммы. Полученный ответ поэтому не хуже оптимума; так как он сам допустим, он равен оптимуму. $\square$

Сложность. Размер каждого списка не превосходит $2^{\lceil n/2 \rceil}$. Перечисление и сортировка, а также бинарные поиски требуют $\mathcal{O}(2^{n/2} \text{poly}(n))$ времени и $\mathcal{O}(2^{n/2})$ памяти. Сравнения считаются точными согласно условию.

5. Задача 5. Минимальная линейная укладка графа

Решение. Для $S \subseteq V$ обозначим число рёбер разреза

$$\text{cut}(S) = |\{\{u, v\} \in E : u \in S, v \notin S\}|.$$

Если перестановка p задаёт последовательность вершин $v_1, \dots, v_n$, а $S_k = \{v_1, \dots, v_k\}$, то

$$\sum_{\{u, v\} \in E} |p_u - p_v| = \sum_{k=1}^{n-1} \text{cut}(S_k). \quad (8)$$

Действительно, ребро с концами в позициях $i < j$ пересекает ровно границы после позиций $i, i+1, \dots, j-1$, то есть учитывается $j-i$ раз.

Пусть $dp[S]$ — минимум суммы $\text{cut}(S_1) + \dots + \text{cut}(S_{|S|})$ по всем порядкам, первые $|S|$ вершин которых образуют S . Полагаем $dp[\emptyset] = 0$. Последней добавленной вершиной может быть любой $v \in S$:

$$dp[S] = \text{cut}(S) + \min_{v \in S} dp[S \setminus \{v\}]. \quad (9)$$

Так как $\text{cut}(V) = 0$, значение $dp[V]$ совпадает с (8). Предки в (9) восстанавливают перестановку.

Доказательство корректности. Тождество (8) доказано подсчётом вклада каждого ребра. В состоянии S последний префикс неизбежно вносит $\text{cut}(S)$, не зависящий от порядка вершин внутри S . После удаления последней вершины v остаётся оптимальная подзадача для $S \setminus \{v\}$; если бы её порядок был неоптимален, его замена улучшила бы весь порядок. Обратно, к оптимальному порядку меньшего множества можно дописать v , получив кандидат (9). Индукция по $|S|$ доказывает точность динамики, а (8) связывает её с исходной целевой функцией. $\square$

Сложность. Все $\text{cut}(S)$ можно вычислить за $\mathcal{O}(2^n(n+m))$ прямым способом либо быстрее рекуррентно. Переходы занимают $\mathcal{O}(n2^n)$. Это $\mathcal{O}(2^n \text{poly}(n, m))$ времени и $\mathcal{O}(2^n)$ памяти.

6. Задача 6. Встречи и настроение

Решение. Ключевой факт: после посещения множества S встреч настроение не зависит от их порядка:

$$m(S) = m_0 + \sum_{i \in S} d_i. \quad (10)$$

Пусть $reachable[S]$ означает, что встречи из S можно посетить в некотором допустимом порядке. База $reachable[\emptyset] = true$. Из достижимого S можно добавить встречу $i \notin S$, если

$$l_i \leq m(S) \leq r_i. \quad (11)$$

Тогда помечаем $S \cup \{i\}$ достижимым. Ответ равен максимальному $popcount(S)$ среди достижимых масок. Суммы $\sum_{i \in S} d_i$ удобно предвычислить рекуррентно по младшему установленному биту.

Доказательство корректности. Каждый переход добавляет встречу, чьё условие (11) выполнено при настроении, которое действительно имеется после уже посещённых встреч по (10). Поэтому любой построенный порядок допустим.

Обратно, возьмём произвольный допустимый порядок множества S и удалим его последнюю встречу i . Префикс порядка доказывает достижимость $S \setminus \{i\}$, а допустимость последнего шага даёт (11) для этого множества. Значит, динамика воспроизводит последний переход. Индукция по числу встреч показывает, что помечаются все и только достижимые множества. Максимальная их мощность и есть искомое число встреч. $\square$

Сложность. Для каждой из 2^n масок перебираются до n встреч: $\mathcal{O}(2^n n)$ времени и $\mathcal{O}(2^n)$ памяти.

7. Задача 7. Замощение доски доминошками

Пусть $n \leq m$; при необходимости доску поворачиваем. Маска длины n описывает клетки текущего столбца, уже занятые горизонтальными домино из предыдущего столбца.

Пункт $\mathcal{O}(4^n(n + m))$

Решение. Для каждой входной маски in рекурсивно заполняем первую свободную клетку столбца. Её можно покрыть вертикальным домино вместе со следующей строкой либо горизонтальным домино, установив соответствующий бит выходной маски out . В худшем случае перебираются $\mathcal{O}(2^n)$ вариантов для каждой из 2^n входных масок. Полученные переходы применяются к каждому столбцу.

Пункт $\mathcal{O}(3^n(n + m))$

Решение. Пару (in, out) можно перечислять тернарной маской: каждая строка находится в одном из состояний «занята слева», «уходит вправо» или «ни то, ни другое». Первые два состояния не могут совпасть. Клетки третьего типа должны разбиваться на соседние вертикальные пары; это проверяется одним проходом. Поэтому всех проверяемых пар не более 3^n . Для каждой допустимой пары выполняем

$$next[out] += cur[in].$$

Пункт $\mathcal{O}(8^n \log m)$

Решение. Построим матрицу переходов T размера $2^n \times 2^n$, где $T_{out,in}$ равно числу способов заполнить один столбец при данных профилях. Тогда число замощений равно

$$(T^m)_{0,0}.$$

Плотное матричное умножение имеет стоимость $(2^n)^3 = 8^n$, а бинарное возведение требует $\mathcal{O}(\log m)$ умножений.

Пункт $\mathcal{O}(2^{nm})$

Решение. Можно не завершать целый столбец одним переходом. Просматриваем nm клеток в фиксированном порядке и храним маску занятости ближайших n клеток. Если текущий младший бит установлен, сдвигаем маску. Иначе пробуем поставить горизонтальное или вертикальное домино, если соответствующая соседняя клетка существует и свободна. На одну пару «позиция — маска» приходится не более двух переходов.

Доказательство корректности. Во всех вариантах используется один и тот же инвариант: профиль содержит точно те ещё не обработанные клетки, которые уже

заняты домино, начатым ранее. Если первая свободная клетка заполняется, домино обязано быть либо вертикальным, либо горизонтальным; иных размещений нет. Оба допустимых варианта рассматриваются и создают правильный новый профиль. Поскольку всегда заполняется первая свободная клетка, одно замощение порождает единственную последовательность переходов, то есть не возникает двойного счёта. Индукция по числу обработанных клеток доказывает точность. Нулевой профиль после последнего столбца означает, что ни одно домино не выходит за границу доски. $\square$

Сложность. Оценки следуют из числа профилей и переходов, указанного в каждом пункте. Память для послойных вариантов $\mathcal{O}(2^n)$, для матричного — $\mathcal{O}(4^n)$.

8. Задача 8. Двухцветные раскраски без монохромных троек

Решение. Кодировем столбец n -битовой маской: 0 и 1 — два цвета. Сначала отметим маски, в которых нет трёх одинаковых последовательных битов; это исключает вертикальные запрещённые тройки.

Чтобы проверить горизонтальные и диагональные тройки, нужно помнить два последних столбца. Состояние после обработки нескольких столбцов есть пара (A, B) . Кандидат C допустим, если:

1. C вертикально допустим;
2. для каждой строки i биты A_i, B_i, C_i не все равны;
3. для всех существующих индексов биты A_i, B_{i+1}, C_{i+2} не все равны;
4. аналогично A_i, B_{i-1}, C_{i-2} не все равны.

Тогда выполняем

$$dp_{\text{new}}[B, C] += dp[A, B]. \quad (12)$$

Для первых одного-двух столбцов проверяются только уже существующие направления. Все допустимые тройки (A, B, C) можно предвычислить.

Доказательство корректности. Любая прямая из трёх клеток, заканчивающаяся в новом столбце C , является вертикальной внутри C , горизонтальной через A, B, C или диагональной через те же три столбца с изменением строки на $+1, +1$ либо $-1, -1$. Перечисленные проверки охватывают ровно эти случаи. Все тройки, целиком лежащие в старых столбцах, уже проверены инвариантом состояния. Следовательно, переход сохраняет допустимость.

Обратно, для любой допустимой раскраски последовательность её столбцовых масок проходит все проверки, поэтому динамика её учитывает. Столбцы определяют раскраску однозначно, значит двойного счёта нет. $\square$

Сложность. Возможны $2^{3n} = 8^n$ тройки масок; проверка одной занимает $\mathcal{O}(n)$. Переходы по m столбцам также можно выполнить, перебирая предвычисленные тройки. Получаем $\mathcal{O}(8^n(n + m))$ времени и $\mathcal{O}(4^n)$ памяти.

9. Задача 9. Разрезание таблицы $2 \times n$ на тройки

Решение. Каждый кусок должен быть связным и содержать три клетки. Его вклад равен 1, если среди трёх значений хотя бы две единицы, и 0 иначе.

Используем профиль постоянной ширины. В момент обработки столбца c храним четырёхбитовую маску занятости клеток в столбцах c и $c + 1$; при выборе фигуры разрешаем также клетки столбца $c + 2$. Берём первую незанятую клетку самого левого ещё не заполненного столбца и перебираем все связные трёхклеточные множества, которые:

- содержат эту клетку;
- лежат в двух строках и столбцах $c, c + 1, c + 2$;
- не пересекаются с уже занятыми клетками и не выходят за доску.

Таких шаблонов постоянное число: горизонтальное 1×3 домино-трио и четыре L -образных подмножества квадрата 2×2 , с возможными сдвигами. После установки фигуры прибавляем её вклад. Когда столбец c полностью заполнен, сдвигаем профиль на один столбец.

Для каждого профиля храним максимальную набранную сумму; недостижимые профили имеют значение $-\infty$. Начальный и конечный профиль нулевые.

Доказательство корректности. В любом разбиении первая ещё не покрытая клетка принадлежит ровно одному связному куску из трёх клеток. Поскольку высота доски равна двум, такой кусок не может иметь горизонтальный размах больше трёх; следовательно, он полностью входит в перечисляемое окно и будет рассмотрен алгоритмом. Удалив этот кусок, получаем подзадачу с новым профилем.

Обратно, каждый переход добавляет связный кусок размера три без пересечений. Требование брать первую незанятую клетку гарантирует, что одна и та же геометрическая разбивка не может быть построена в другом порядке. Индукция по числу установленных кусков показывает взаимно однозначное соответствие между полными путями динамики и допустимыми разбиениями. Поскольку вклад каждого куска прибавляется ровно при его установке, максимизация динамики совпадает с условием. $\square$

Сложность. Число профилей и шаблонов постоянно, поэтому каждый столбец обрабатывается за $\mathcal{O}(1)$. Время $\mathcal{O}(n)$, память $\mathcal{O}(1)$.

10. Задача 10. Покупка рюкзаков

Пусть $w(S) = \sum_{i \in S} c_i$, а

$$price[x] = \min\{\text{цена модели вместимости не меньше } x\}.$$

Если подходящей модели нет, $price[x] = +\infty$.

Пункт $\mathcal{O}(2^n m + 3^n)$

Решение. Для каждой маски S вычислим $w(S)$, затем просмотром всех m моделей найдём $price[w(S)]$. Пусть $dp[S]$ — минимальная стоимость рюкзаков для предметов из S . Тогда

$$dp[\emptyset] = 0, \quad dp[S] = \min_{\emptyset \neq T \subseteq S} (dp[S \setminus T] + price[w(T)]). \quad (13)$$

Чтобы не перебирать один и тот же неупорядоченный разрез лишний раз, можно требовать, чтобы T содержало младший установленный бит S , но для оценки это несущественно.

Доказательство корректности. В любом оптимальном решении выберем рюкзак, содержащий множество T предметов. Минимальная возможная цена рюкзака для него равна $price[w(T)]$, а оставшиеся предметы требуют не меньше $dp[S \setminus T]$. Поэтому решение не дешевле правой части (13). Обратно, каждый кандидат (13) объединяет оптимальную упаковку дополнения с одним рюкзаком, вмещающим T , и потому допустим. Минимум равен оптимуму. $\square$

Пункт $\mathcal{O}(2^n nC + m)$

Решение. Пусть C — максимальная вместимость. Сначала для каждой точной вместимости сохраняем минимальную цену модели, затем одним проходом справа налево строим суффиксные минимумы $price[x]$. Это занимает $\mathcal{O}(C + m)$.

Ускорим перебор T . Для каждой маски S и $0 \leq x \leq C$ поддержим

$$g[S][x] = \min_{\substack{R \subseteq S \\ w(S \setminus R) = x}} dp[R]. \quad (14)$$

Маски просматриваются по возрастанию числа битов. После вычисления $dp[S]$ задаём $g[S][0] \leftarrow \min(g[S][0], dp[S])$. Затем для каждого $i \notin S$ и $x + c_i \leq C$ распространяем

$$g[S \cup \{i\}][x + c_i] \leftarrow \min(g[S \cup \{i\}][x + c_i], g[S][x]). \quad (15)$$

К моменту обработки S все значения с $x > 0$ уже пришли из меньших масок. Поэтому

$$dp[S] = \min_{1 \leq x \leq C} (g[S][x] + price[x]), \quad dp[\emptyset] = 0. \quad (16)$$

Доказательство корректности. Докажем инвариант (14). Источник $g[R][0] = dp[R]$ представляет выбор пустого множества $S \setminus R$. Каждый переход (15) добавляет ровно один предмет в $S \setminus R$ и увеличивает его суммарный вес на вес предмета. Любая

пара $R \subseteq S$ получается единственным набором таких добавлений (порядков может быть несколько, но операция минимума устраняет повторы), а никакая иная пара не возникает. Следовательно, после всех меньших масок $g[S][x]$ имеет точный смысл (14).

В формуле (16) множество $S \setminus R$ есть содержимое последнего рюкзака, x — его вес, а $dp[R]$ — цена остальных рюкзаков. Таким образом, (16) является той же оптимальной декомпозицией, что и (13), но кандидаты сгруппированы по весу. Корректность следует из уже доказанной корректности (13). $\square$

Сложность. В первом пункте подготовка занимает $\mathcal{O}(2^n m)$, а число пар (S, T) , $T \subseteq S$, равно 3^n ; память $\mathcal{O}(2^n)$. Во втором пункте каждое состояние (S, x) распространяется не более чем по n предметам: $\mathcal{O}(2^n n C + m + C)$ времени и $\mathcal{O}(2^n C)$ памяти.

11. Задача 11. Хроматическое число

Пункт $\mathcal{O}(3^n)$

Решение. Предвычислим булево $independent[S]$: является ли S независимым множеством. Это делается за $\mathcal{O}(n2^n)$, например удалением младшей вершины и проверкой отсутствия её соседей.

Пусть $dp[S]$ — хроматическое число индуцированного графа $G[S]$. Тогда

$$dp[\emptyset] = 0, \quad dp[S] = 1 + \min_{\substack{\emptyset \neq I \subseteq S \\ independent[I]}} dp[S \setminus I]. \quad (17)$$

Доказательство корректности. Любой цветовой класс есть независимое множество I . Удалив один класс из оптимальной раскраски S , получаем раскраску $S \setminus I$, которой нужно не меньше $dp[S \setminus I]$ цветов; это даёт нижнюю границу (17). Обратно, независимое I можно покрасить одним новым цветом, а дополнение — оптимально по динамике, поэтому каждый кандидат достижим. Значит, минимум точен. $\square$

Пункт $\mathcal{O}(2^n \text{poly}(n))$

Решение. Для каждого $S \subseteq V$ найдём число $I(S)$ независимых подмножеств в $G[S]$, включая пустое. Если $v \in S$, то

$$I(S) = I(S \setminus \{v\}) + I(S \setminus (\{v\} \cup N(v))), \quad I(\emptyset) = 1. \quad (18)$$

Граф k -раскрашиваем тогда и только тогда, когда существуют k независимых множеств, объединение которых равно V . Число упорядоченных k -кортежей таких множеств по включениям-исключениям равно

$$C_k = \sum_{S \subseteq V} (-1)^{n-|S|} I(S)^k. \quad (19)$$

Следовательно, G k -раскрашиваем ровно тогда, когда $C_k > 0$. Проверяем $k = 1, \dots, n$ и возвращаем первый подходящий.

Доказательство корректности. Формула (18) делит независимые подмножества на не содержащие v и содержащие v ; во втором классе запрещены все соседи v .

Для (19) рассмотрим все k -кортежи независимых подмножеств. Для фиксированного S величина $I(S)^k$ считает кортежи, каждый компонент которых лежит в S . Включения-исключения по вершинам, не покрытым объединением компонент, оставляют ровно кортежи с объединением V . Их существование эквивалентно k -раскрашиваемости: цветовые классы дают такой кортеж; обратно, если независимые множества покрывают V , каждой вершине назначаем любой индекс множества, которое её содержит. Полученные классы являются подмножествами независимых множеств и потому независимы. Итак, критерий $C_k > 0$ точен, а первый подходящий k есть хроматическое число. $\square$

Сложность. В (17) суммарное число пар $I \subseteq S \subseteq V$ равно 3^n . Для второго способа все $I(S)$ вычисляются за $\mathcal{O}(2^n)$ переходов после хранения соседей масками; проверка всех

$k \leq n$ занимает $\mathcal{O}(n2^n \text{polylog } n)$ арифметических операций. Это $\mathcal{O}(2^n \text{poly}(n))$ времени и $\mathcal{O}(2^n)$ памяти. Как оговорено в листке, операции с большими целыми считаются за $\mathcal{O}(1)$.

12. Задача 12. Минимальное число пакетов

Пусть вместимость пакета равна s , веса предметов $w_i \leq s$.

Пункт $\mathcal{O}(4^n)$

Решение. Для каждой маски заранее проверим, помещается ли она в один пакет. Затем для каждого S можно перебрать все 2^n масок T , проверить $T \subseteq S$ и допустимость T , после чего обновить

$$dp[S] = \min(dp[S], 1 + dp[S \setminus T]).$$

Это даёт $2^n \cdot 2^n = 4^n$.

Пункт $\mathcal{O}(3^n)$

Решение. Перебираем только подмаски $T \subseteq S$. Суммарное число таких пар равно

$$\sum_{S \subseteq V} 2^{|S|} = (1 + 2)^n = 3^n.$$

Можно также требовать, чтобы T содержало фиксированный бит S , не меняя асимптотику.

Пункт $\mathcal{O}(2^n n)$

Решение. Для каждой маски храним пару

$$dp[S] = (\text{число уже открытых пакетов}, \text{вес в последнем пакете}),$$

сравниваемую лексикографически. Начинаем с $dp[\emptyset] = (1, 0)$. Из S добавляем предмет $i \notin S$:

$$candidate = \begin{cases} (b, x + w_i), & x + w_i \leq s, \\ (b + 1, w_i), & x + w_i > s, \end{cases}$$

где $dp[S] = (b, x)$, и минимизируем $dp[S \cup \{i\}]$.

Доказательство корректности. Первые два варианта корректны по той же декомпозиции по последнему пакету, что и (17): его содержимое является допустимой подмаской, а дополнение упаковывается оптимально.

Для третьего варианта докажем инвариант: $dp[S]$ лексикографически минимально среди всех упаковок предметов S , если пакеты рассматриваются в порядке их открытия. Любой такой порядок имеет последний добавленный предмет i ; до него была некоторая упаковка $S \setminus \{i\}$. Если предмет помещается в последний пакет, открывать новый невыгодно; если не помещается, новый пакет необходим. При одинаковом числе пакетов меньшая загрузка последнего пакета никогда не ухудшает будущие возможности, поэтому достаточно хранить только лучшую пару. Индукция по $|S|$ доказывает инвариант. Первая координата $dp[V]$ есть минимум пакетов. $\square$

Сложность. Получаются соответственно $\mathcal{O}(4^n)$, $\mathcal{O}(3^n)$ и $\mathcal{O}(2^n n)$ времени. В каждом варианте память $\mathcal{O}(2^n)$.

13. Задача 13. Интересные тройки шестигранных кубиков

Модель. Каждый кубик задаётся неубывающей шестёркой значений:

$$A = (A_1 \leq \dots \leq A_6),$$

что взаимно однозначно мультимножеству на его гранях. Для пары кубиков A, B обозначим

$$w(A, B) = |\{(i, j) : A_i > B_j\}|.$$

Условие A превосходит B эквивалентно $w(A, B) \geq 19$.

Пункт $\mathcal{O}(n)$

Решение. Просматриваем возможные значения $x = 1, \dots, n$. Состояние содержит

$$(i, j, k, p, q, r),$$

где $i, j, k \in \{0, \dots, 6\}$ — уже размещённые грани A, B, C , а

$$p = w(A, B), \quad q = w(B, C), \quad r = w(C, A)$$

для уже обработанных значений.

На уровне x выбираем числа α, β, γ граней A, B, C , равных x , в пределах оставшихся количеств. Допускается и $(0, 0, 0)$. Если старое состояние содержит i, j, k , то

$$\begin{aligned} i' &= i + \alpha, & p' &= p + \alpha j, \\ j' &= j + \beta, & q' &= q + \beta k, \\ k' &= k + \gamma, & r' &= r + \gamma i. \end{aligned} \tag{20}$$

Например, каждая новая грань A со значением x превосходит ровно j уже размещённых, то есть строго меньших, граней B ; равные x грани не учитываются. После n уровней суммируем состояния

$$i = j = k = 6, \quad p, q, r \geq 19.$$

Доказательство корректности. Неубывающая шестёрка каждого кубика однозначно задаётся количеством его граней, равных каждому x . Поэтому последовательность выборов (α, β, γ) взаимно однозначно соответствует упорядоченной тройке мультимножеств.

При обработке значения x новые строгие победы возникают только между новой гранью и ранее обработанной, строго меньшей гранью соперника. Формулы (20) учитывают каждую такую пару ровно в момент появления её большей грани; пары равных значений не учитываются. Индукция по x доказывает точность всех трёх счётчиков. Финальные пороги 19 в точности выражают три вероятности, строго большие $1/2$, поэтому сумма принимающих состояний равна ответу. $\square$

Пункт $\mathcal{O}(\log n)$

Решение. Множество состояний и переходов (20) не зависит от n и имеет постоянный размер. Пронумеруем состояния и построим постоянную матрицу M , где M_{uv} равно числу выборов (α, β, γ) , переводящих состояние v в u за одно значение x . Тогда вектор после всех значений равен

$$v_n = M^n v_0.$$

Бинарное возведение M в степень даёт $\mathcal{O}(\log n)$ арифметических операций с постоянным, хотя и большим, множителем.

Пункт $\mathcal{O}(1)$

Решение. Удалим из описания уровни, на которых $(\alpha, \beta, \gamma) = (0, 0, 0)$. Получается *шаблон относительного порядка*: последовательность

$$(\alpha_1, \beta_1, \gamma_1), \dots, (\alpha_h, \beta_h, \gamma_h),$$

в которой каждая тройка ненулевая, а суммы по каждой координате равны 6. Число уровней h лежит между 1 и 18. По тем же формулам (20) можно один раз, независимо от n , вычислить константу c_h — количество интересных шаблонов с h различными значениями.

Для реализации фиксированного шаблона нужно выбрать из $\{1, \dots, n\}$ h различных значений; после сортировки они единственным образом назначаются уровням шаблона. Число выборов равно $\binom{n}{h}$. Следовательно,

$$\boxed{\text{ans}(n) = \sum_{h=1}^{18} c_h \binom{n}{h}}. \quad (21)$$

Константы c_h предвычисляются один раз и могут быть зашиты в программу. В каждом запросе восемнадцать биномиальных коэффициентов вычисляются последовательно:

$$\binom{n}{0} = 1, \quad \binom{n}{h+1} = \binom{n}{h} \frac{n-h}{h+1}.$$

Доказательство корректности. Отбрасывание пустых уровней оставляет только строгий порядок различных значений граней и все равенства внутри каждого уровня. Такой шаблон однозначно определяет все попарные сравнения, а значит, и интересность тройки. Обратно, любая тройка кубиков порождает единственный шаблон после сжатия её различных значений в $1, \dots, h$.

Для каждого интересного шаблона выбор h фактических значений из $[1, n]$ полностью и однозначно восстанавливает тройку кубиков. Шаблоны с разными h или разной последовательностью не пересекаются. Поэтому правило произведения и последующее суммирование дают (21). $\square$

Сложность. Число допустимых состояний, переходов и $h \leq 18$ является абсолютной константой. Первый алгоритм имеет время $\mathcal{O}(n)$, второй — $\mathcal{O}(\log n)$, третий после постоянного предвычисления — $\mathcal{O}(18) = \mathcal{O}(1)$. Память во всех трёх оценках $\mathcal{O}(1)$ относительно n . Как и разрешено условием, арифметические операции с ответами считаются за $\mathcal{O}(1)$.